

Repository

Repository Pattern

Collection-like access to aggregates

DEFINITION

Mediate between the domain and data layers, exposing a collection-like interface for accessing aggregates.

A Repository is an in-memory illusion of a collection of aggregates: the domain calls find / add / remove in its own language and never sees SQL. One repository serves one aggregate root, and all the mapping to tables, joins and the ORM hides behind that port — so persistence becomes a detail the domain can ignore.

WHY IT MATTERS

If domain code writes queries, persistence concerns smear across the whole codebase and you can't exercise business logic without a database. A repository concentrates that knowledge behind one interface: the domain stays pure and testable against an in-memory fake, and every query for an aggregate lives in exactly one place to tune.

— How it works

Repository

A collection-like interface in domain terms — byld, save, the queries the domain needs. The port the domain depends on.

Aggregate

The unit a repository stores and returns whole — a consistency boundary (an Order with its lines), not a single table row.

Mapper / ORM

The implementation behind the port — TypeORM, a data mapper — translating aggregates to rows and back. Swappable and fakeable.

HOW TO APPLY

1. Define one interface per aggregate root, in domain language (OrderRepo).
2. Return whole aggregates, not table rows or ORM entities.
3. Put all SQL/ORM code in the implementation, behind the interface.
4. Inject the repository so tests use an in-memory fake and prod uses the ORM.

LITMUS TEST

Can you run your domain logic against an in-memory fake with zero SQL? If business code contains query strings, the repository boundary is leaking.

— Smell → fix

A domain service hand-writing SQL and mapping rows itself:

```
class CheckoutService {
  pay(id: string) {
    const row = db.query('select * from orders where id=?', id)
    const order = mapRow(row) // mapping inside the domain
  }
} // SQL + business rules tangled together
```

↓ HIDE STORAGE BEHIND A PORT

```
interface OrderRepo {
  byId(id: string): Promise<Order>
  save(o: Order): Promise<void>
}
// SqlOrderRepo implements it via TypeORM, behind the port
// domain talks to OrderRepo only: swappable + fakeable
```

The domain depends only on OrderRepo. Tests inject an in-memory fake; production wires SqlOrderRepo (TypeORM) behind the same port. Resist a generic find-everything repo — expose intention-revealing methods like openOrdersFor(customer).

USE WHEN

- ✓ Keep persistence out of the domain
- ✓ Swap the DB or fake it in tests

AVOID WHEN

- ▲ Thin CRUD apps where a generic repo becomes a leaky abstraction

— Trade-offs & pitfalls

PROS

- + Clean, testable domain
- + Centralized query logic

CONS

- Anemic generic repos
- Leaky, ad-hoc query methods

COMMON PITFALLS

- ▲ A generic Repository<T> that leaks CRUD and invites ad-hoc queries everywhere.
- ▲ Returning ORM entities with lazy-loading, leaking persistence into the domain.
- ▲ One repository per table instead of per aggregate — that's a DAO, not a Repository.

WHERE YOU SEE IT

- ▶ Spring Data, TypeORM custom repositories, EF Core DbSet behind an interface.
- ▶ DDD codebases where each aggregate root gets its own repository.
- ▶ A hexagonal-architecture "port" with an in-memory adapter for tests.

SEE ALSO

Unit of Work	repositories enlist their new/dirty/removed objects; the Unit of Work commits them in one transaction
DIP	the domain owns the OrderRepo interface; the SQL implementation depends inward on it
Dependency Injection	the concrete repo is injected, which is what makes storage swappable and fakeable
Facade	a repository is a domain-facing facade over the ORM / data mapper

— Interview & sources

Repository vs DAO?

A DAO is table/row-oriented and close to the database; a Repository is aggregate/domain-oriented and speaks the ubiquitous language. One repository per aggregate root, not one per table.

Is a generic Repository<T> a good idea?

Usually a smell: it leaks CRUD and invites ad-hoc queries everywhere. Prefer explicit, intention-revealing methods per aggregate; add a generic base only for genuinely shared behaviour.

Where does query logic live?

Inside the repository implementation, never the domain. The interface stays in domain terms; the impl translates to SQL/ORM and is the single place to tune queries.

How do you handle complex reporting through a Repository?

Keep the aggregate repository for transactional domain access, and use a separate read model (CQRS) or query service for reporting. Forcing every report through the aggregate repository bloats it — reads and writes have different shapes.

REMEMBER

Talk to storage in domain words — "find the order", not "SELECT * FROM orders".

SOURCES

Eric Evans — "Domain-Driven Design" (2003): Repositories for aggregate access

Martin Fowler — "Patterns of Enterprise Application Architecture" (2002): Repository

TypeORM, EF Core, Hibernate — repository & data-mapper implementations